

How I run an AI agent team you can trust.

1 · RIGHT MODEL FOR EACH ROLE

Routing is config, not habit

Implementation & delegated tasks **GPT-6.1 Sol**

Architecture, root cause, independent review **Claude Opus 5.5**

Lookups, titles, memory, commit messages **GPT-6 Luna**

Parallel subagents **up to 32**

Subscription logins only. No API keys for coding agents, no silent fallback to another provider.

2 · CLEAR DECISION RIGHTS

Agents act. I approve what can't be undone.

AGENT Design, root cause, refactors, tests, Issues, branches, PRs, review requests: decide, act, report why

ME Deploys, store submissions, payments, production data, external messages, credentials: one ask, everything prepared up to approval

ME Product direction, pricing, final Korean copy

3 · TWO SESSIONS PER CHANGE

No agent grades its own work

1 Author agent
sole writer of the branch, tests and evidence

2 Reviewer agent
separate read-only session, verdict on the exact commit

3 Integration
merge pinned to the reviewed commit; merge is not deploy

19% sent back by review
0 / 231 merged PRs reverted

REAL CATCH

A reviewer agent **blocked a production bug fix** that would have sent users' birth details to the error tracker. The author agent fixed it and added a regression test in **1 h 16 m**; merged the same night.

The reviewer agent stopped a privacy leak before it reached production.

A production bug fix, an independent read-only review, and the fix that made it safe. Times in KST.

ME

Owner Sep 13 · 21:21

PO BUG

Production alert: valid chart requests fail for later dates, so users get no chart.

AU

Author agent 23:40

Opened a PR: extend the calculation range and improve error reporting.

RV

Reviewer agents code + architecture, read-only · 00:15

REQUEST CHANGES

The new error reports would send users' **birth date, time and timezone** to the external error tracker.

AU

Author agent 01:31

Kept personal details out of error reports and added a **regression test** that proves none reach the tracker.

RV

Reviewer agents 02:06

APPROVE

Re-ran tests and lint on the exact merge result. Merged.

VF

Verification next day

VERIFIED

Deployed build contains the fix; both error groups went quiet.

TIMING

Sent back to fixed	1 h 16 m
Fix to approval	35 m
Bug report to merge	4 h 45 m

WHY IT MATTERS

The bug was real and the fix was urgent. Only a **separate session with a different job** stopped to ask what the new error reports would contain. That is the point of never letting an agent grade its own work.

ACROSS SEP TO OCT 2026

PRs sent back first	19%
Merged PRs reverted	0 / 231

One week of agent work

Sep 28 to Oct 4, 2026 · TriAstra (private repo, titles generalized) · KST

1 block = 1 merged PR · **red outline** = sent back by review first



78

PRs merged

11

sent back by review

0

reverted

17 min

median open to merge

27

issues closed

3

closed, not merged

WHAT GOT MERGED

Refactor 28

Infra 17

Feature  14

Category	Count
Bug fix	11

Tests 7

Docs 1

WHO DID WHAT

Author agents wrote every PR. A separate read-only reviewer agent gave each one a verdict on the exact commit. I set priorities and approved deploys.

ONE ENGINEER, AN AGENT TEAM

Mingyun Chae

TriAstra · Jan 2026 to today

Every role of a product team, covered by me and my AI agents.

Product & design

ME

Specs, UX, acceptance criteria

Backend

ME

AUTHOR AGENTS

Rust/Axum + PostgreSQL I own the architecture

Web & ChatGPT app

AUTHOR AGENTS

React web, admin, ChatGPT app 7 languages

Code review

REVIEWER AGENT

Independent, read-only sent back 19% of PRs

QA

VERIFIER AGENT

Reruns every check 3,600+ server tests

DevOps

ME

AGENTS

Terraform on GCP + Cloudflare only a dispatcher can deploy

Ops, support, growth

ME

BROWSER AGENT

Payments, support, launch

1,600+

registered users

231

merged PRs, 0 reverted

1.9 h

median, bug report to fix

2

flat-rate AI subscriptions

One engineer on a client's codebase: 702 PRs merged in 30 days.

Mingyun Chae

Client project · Singapore B2B platform

Sep 2026 · on-site contract

THE ISSUE PIPELINE I SET UP

- 1 Dispatcher picks the next issue**
when a slot frees, by priority; one worktree per issue
- 2 Worker agent: plan, build, test, PR**
can stop and ask; never judges or merges its own PR
- 3 Reviewer agent in a separate process**
sees only the PR and gate logs, not the worker's session

APPROVE → merge train

CHANGES → worker again

HOLD → my decision

- 4 Deploys only in a dedicated session**
an advisor model watches every turn; risky commands are blocked before they run

I only see what needs a human: **held PRs and stopped issues**, collected in one decision queue.

Built mid-contract: live from Sep 23, after which **422 of the 702 PRs** were merged through the independent reviewer.

RESULTS

702

PRs merged in 30 days, median 1.3 h to merge

6 days

all 15 prototype screens made real across 3 role portals

15 + 30

unauthenticated APIs locked; write APIs given role checks

156

legacy server templates removed (11.5k lines)

2 to 4 min

pre-merge E2E smoke gate (78 tests) on every PR, instead of the 33-minute full suite

5.0 / 5

client rating, as the sole engineer on an existing Django + React platform